



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА



Кафедра цифровой трансформации

Институт информационных технологий

Презентации по лекционным материалам по дисциплине «Разработка баз данных»

Направления подготовки:

Уровень: бакалавриат

Форма обучения: очная

01.03.04 Прикладная математика
09.03.03 Прикладная информатика
09.03.04 Программная инженерия
09.03.01 Информатика и вычислительная техника

Лекция №6. Триггеры и курсоры

Лектор
Исаева Мария Владимировна
Кандидат технических наук,
доцент кафедры
цифровой трансформации

2025/2026 учебный год

СВЕДЕНИЯ О ДИСЦИПЛИНЕ

Лекции ведут:

- Исаева Мария Владимировна, к.т.н., доцент каф. Цифровой трансформации
- Дзгоев Алан Эдуардович, к.т.н., доцент каф. Цифровой трансформации
(Dzgoviev@mirea.ru / При обращении в теме письма указывайте номер группы)
- Миронов Антон Николаевич, Старший преподаватель каф. Цифровой трансформации
- Семыкина Наталья Александровна, д.т.н., профессор каф. Цифровой трансформации
- Резеньков Роман Николаевич, к.т.н., доцент каф. Цифровой трансформации

Объём **аудиторной** работы по дисциплине в 5 семестре:

- Лекции – 16 часов;
- Практические занятия – 48 часа;
- Аттестация – экзамен.

Допуск к экзамену:

- посещение лекций и практических работ;
- выполнение в установленный срок всех практических работ.

подробности в памятке по дисциплине (следующие слайды)

Выполнение практических заданий

1. Все материалы курса на <https://online-edu.mirea.ru> в разделе **Разработка баз данных: доступ к материалам курса открывается по расписанию занятий.**

2. Доступ с аккаунта МИРЭА.

3. Все создаваемые файлы прикладываются к заданию.

4. Учёт выполненных работ.

5. Общение с преподавателем через отзывы в заданиях.

Курс	Л4_ % кликов по кнопке "контроль присутствия"	Л5_ % кликов по кнопке "контроль присутствия"	Л6_04-05_ % присутствия	Л7_18-05_ % присутствия	Л8_01-06_ % присутствия	%СРЗНАЧ участия в лекциях	Задание:Практика 1_Законодательная и нормативно-техническая база стандартизации в РФ	Задание:Практика 2_Нормативные документы по стандартизации ИТ	Задание:Практика 3_ч1_Классификаторы в области ИТ	Задание:Практика 3_ч2_Классификаторы в области ИТ	Задание:Практика 3_ч3_Классификаторы в области ИТ	Задание:Практика 4_ч1_Создание технического задания в соответствии с ГОСТ 34.602-2020
0	0	50				6,3	-	-	-	-	-	-
0	0	50		100	100	31,3	-	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
00	0	50	100		100	56,3	1	1	1	1	1	1
100	100	100	100	100	100	87,5	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	50	0				6,3	1	-	-	-	-	-
100	0	100		100		56,3	1	1	1	1	1	1
0	0	0				12,5	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	-	-	-	-	-	-
0	100	100	100	100	100	68,8	1	1	1	1	1	1
0	0	50				18,8	-	-	-	-	-	-
0	0	0	100	100	100	50	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	100	100	100	100	100	100	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	1	1	1	1	-	-
50	0	50				18,8	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0			100	12,5	1	1	1	1	1	1
100	100	0				25	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	100	100	100	100	100	100	1	1	1	1	1	1



Рекомендуемая литература

1. Новиков Б.А. Основы технологий баз данных: учебное пособие / Б.А. Новиков, Е.А. Горшкова, Н.Г. Графеева; под.ред. Е.В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. Режим доступа: <https://postgrespro.ru/education/books/dbtech>
2. Моргунов Е.П. PostgreSQL. Основы языка SQL: учеб. Пособие / Е.П. Моргунов; под. ред. Е.В. Рогова, П.В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.: ил. Режим доступа: <https://postgrespro.ru/education/books/sqlprimer>
3. Комаров В.И. Путеводитель по базам данных. – М.: ДМК-Пресс, 2024. – 520 с. Режим доступа: <https://edu.postgrespro.ru/dbguide.pdf>
4. Смирнов М.В. Проектирование баз данных [Электронный ресурс]: Конспект лекций / Смирнов М.В. - М., МИРЭА – Российский технологический университет, 2020 – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/163892/>
5. Чистякова М.А. Проектирование и эксплуатация баз данных [Электронный ресурс]: Учебно-методическое пособие / Чистякова М.А., Иванова И.А., Котилевец И.Д. – М.: МИРЭА – Российский технологический университет, 2021. – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/176572/>
6. Братусь Н.В. Базы данных. Часть 1 [Электронный ресурс]: Практикум / Братусь Н.В., Маличенко С.В., Матчин В.Т. – М.: МИРЭА – Российский технологический университет, 2024. – 1 электрон. опт. диск (CD-ROM) – Режим доступа: <https://ibc.mirea.ru/books/SHARE/5859/>
7. Моргунов Е. П. PostgreSQL. Профессиональный SQL : учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова. – М.: ДМК Пресс, 2025. – 444 с. Режим доступа: <https://postgrespro.ru/education/books/advancedsql>
8. Рогов Е. В. PostgreSQL 17 изнутри. – М.: ДМК Пресс, 2025. – 668 с. Режим доступа: <https://postgrespro.ru/education/books/internals>
9. Введение в системы баз данных. : Пер. с англ. / К. Дж. Дейт .— М. : Изд. дом "Вильямс", 2001 .— 1072 с. https://ibc.mirea.ru/books/search/?search_field=Дейт&page=3

Неделя	Лекции	Практические работы	Дедлайны, текущие и контрольные мероприятия
1	Лекция 1. Тема: Реляционная модель данных и базовые операции SQL. (2 часа)	Занятие 1. ПРАКТИЧЕСКАЯ РАБОТА №1. Создание БД и запросы на выборку данных (2 часа)	
2		Занятие 2. ПРАКТИЧЕСКАЯ РАБОТА №1. (2 часа)	Дедлайн по защите отчета по практике №1.
		Занятие 3. ПРАКТИЧЕСКАЯ РАБОТА №2. Многотабличные запросы, использование операций объединения, пересечения, разности (2 часа)	
3	Лекция 2. Тема: Многотабличные запросы и теоретико-множественные операции. (2 часа)	Занятие 4. ПРАКТИЧЕСКАЯ РАБОТА №2. (2 часа)	
4		Занятие 5. ПРАКТИЧЕСКАЯ РАБОТА №2. (2 часа)	Дедлайн по защите отчета по практике №2.
		Занятие 6 ПРАКТИЧЕСКАЯ РАБОТА №3 Использование подзапросов и CTE (2 часа)	
5	Лекция 3. Использование подзапросов и объединения выражения (2 часа)	Занятие 7. ПРАКТИЧЕСКАЯ РАБОТА №3 (2 часа)	Тест №1. (по лекциям №1-2).
6		Занятие 8. ПРАКТИЧЕСКАЯ РАБОТА №3 (2 часа).	Дедлайн по защите отчета по практике №3.
		Занятие 9. ПРАКТИЧЕСКАЯ РАБОТА №4 Использование оконных функций и функций ранжирования (2 часа)	
7	Лекция 4. Тема: SQL. Оконные функции и аналитические запросы (2 часа)	Занятие 10. ПРАКТИЧЕСКАЯ РАБОТА №4 (2 часа)	Дедлайн по защите отчета по практике №4.
8		Занятие 11. ПРАКТИЧЕСКАЯ РАБОТА №4 (2 часа)	Дедлайн по защите отчета по практике №4.
		Занятие 12. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	
9	Лекция 5. Операторы модификации данных, объекты базы данных (2 часа)	Занятие 13. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	Дедлайн по защите отчета по практике №5.

План – график лекций и практических занятий

10		Занятие 14. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	Дедлайн по защите отчета по практике №5.
		Занятие 15. ПРАКТИЧЕСКАЯ РАБОТА №6 (2 часа)	
11	Лекция 6. Управление данными: триггеры, курсоры (2 часа)	Занятие 16. 2 часа	Контрольная работа №1.
12		Занятие 17. ПРАКТИЧЕСКАЯ РАБОТА №6 (2 часа)	Дедлайн по защите отчета по практике №6.
		Занятие 18. ПРАКТИЧЕСКАЯ РАБОТА №7 Оптимизация запросов (2 часа)	Тест №2. (по лекциям №3-5).
13	Лекция 7. Оптимизация запросов (2 часа)	Занятие 19. ПРАКТИЧЕСКАЯ РАБОТА №7 (2 часа)	
14		Занятие 20. ПРАКТИЧЕСКАЯ РАБОТА №7 (2 часа)	Дедлайн по защите отчета по практике №7.
		Занятие 21. ПРАКТИЧЕСКАЯ РАБОТА №8 Хранение неструктурированных данных (2 часа)	
15	Лекция 8. Администрирование баз данных и хранение неструктурированных данных (2 часа) Тест №3. (по лекциям №6-7).	Занятие 22. ПРАКТИЧЕСКАЯ РАБОТА №8 (2 часа)	
16		Занятие 23. ПРАКТИЧЕСКАЯ РАБОТА №8 (2 часа)	Дедлайн Практики №8.
		Занятие 24. 2 часа	Контрольная работа №2.
Итого	16 часов	48 часов	

ПАМЯТКА

о правилах обучения дисциплины

«Разработка баз данных» (2025-2026, I семестр)

№	Наименование	Формат	Период проведения	Баллы (макс.)
1	Защита практических работ (8 практик)	Очно	Сентябрь 2025 – Декабрь 2025	48 (6 баллов за 1 практику)
2	Контрольный тест №1 (по лекциям №1 и №2)	Очно, СДО	Октябрь 2025	5
3	Контрольный тест №2 (по лекциям №3 - №5)	Очно, СДО	Ноябрь 2025	5
4	Контрольная работа №1	Очно, СДО	Ноябрь 2025	10
5	Контрольный тест №3 (по лекциям №6 - №8)	Очно, СДО	Декабрь 2025	6
6	Контрольная работа №2	Очно, СДО	Декабрь 2025	10
7	Посещение (лекции 8 занятий, практики 24 занятия)	Очно	Сентябрь– Декабрь 2025	16 б. 0,5 баллов за посещения 1 лекции и практики (0,5·32 пар)
Максимальная сумма баллов за активность по дисциплине в течение учебного семестра				100

ДИСЦИПЛИНА	Разработка баз данных (укажите полное наименование дисциплины без сокращений)
ИНСТИТУТ	информационных технологий (укажите название учебного института)
КАФЕДРА	Цифровой трансформации (укажите полное наименование кафедры)
Уровень обучения	Бакалавриат
Аудиторные часы	16/0/48 (укажите количество часов лекционных, лабораторных и практических)

Лекции 16 часов.

Обязательное посещение всех лекций.

В тестах в рамках мероприятий текущего контроля вопросы из лекций

Практические работы (8 работ)

Обязательное посещение всех практических занятий.

Допуск к экзамену – очная защита всех практических работ в течение семестра. Защита отчёта до дедлайна

Защита практической работы после дедлайна – 0 баллов, но работа засчитывается.

Если есть справка по перенесённой болезни, заверенная в учебном отделе, то эту справку необходимо принести и показать преподавателю по практике. В таком случае назначается пересдача пропущенной практической работы в день консультаций (важно: студент защищает именно ту практическую работу, которую он пропустил по болезни, согласно датам в справке). Если студент пропустил практические работы без уважительной причины и, соответственно, работу не защитил, то на экзамен он не допускается.

Если студент загрузил отчет до дедлайна, но не защитил, то такой отчет не засчитывается студенту, т.е. не сдал.

Дополнительного времени на защиту практических работ не выделяется.

Выполненные работы студент загружает в СДО в формате pdf. Фон области построения модели – белый (в отчёте).

ПАМЯТКА

о правилах обучения дисциплины

«Разработка баз данных» (2025-2026, I семестр)

№	Наименование	Формат	Период проведения	Баллы (макс.)
1	Защита практических работ (8 практик)	Очно	Сентябрь 2025 – Декабрь 2025	48 (6 баллов за 1 практику)
2	Контрольный тест №1 (по лекциям №1 и №2)	Очно, СДО	Октябрь 2025	5
3	Контрольный тест №2 (по лекциям №3 - №5)	Очно, СДО	Ноябрь 2025	5
4	Контрольная работа №1	Очно, СДО	Ноябрь 2025	10
5	Контрольный тест №3 (по лекциям №6 - №8)	Очно, СДО	Декабрь 2025	6
6	Контрольная работа №2	Очно, СДО	Декабрь 2025	10
7	Посещение (лекции 8 занятий, практики 24 занятия)	Очно	Сентябрь– Декабрь 2025	16 б. 0,5 баллов за посещения 1 лекции и практики (0,5·32 пар)
Максимальная сумма баллов за активность по дисциплине в течение учебного семестра				100

Если студент не загрузил отчет до дедлайна, то загрузка после дедлайна станет недоступна.

Контрольные тесты

Тесты студенты выполняют в СДО на паре.

Баллы минусуются, если по базовым вопросам студент отвечает не правильно.

Проходного балла нет, сколько студент набрал столько и остается.

Контрольный тест №1. (Лекции 1, 2) проводится на 5-м практическом занятии в начале пары (20 минут).

Контрольный тест №2. (Лекции 3, 4, 5) проводится на 16-м практическом занятии в начале пары (20 минут)..

Контрольный тест №3 (Лекции №6-7) проводится на 8-й лекции в начале пары (20 минут).

Контрольные работы

Выполняются на 15 и 24 практических занятиях в аудитории.

Студент получает задание на паре.

Время выполнения контрольной работы – 1 час.

Экзамен

Допуск к экзамену – защита всех практических работ в течение семестра очно. В случае, если студент не сдал какие-либо практики, у него есть возможность сдать их во время экзамена, если успеет. В противном случае, отправляется на пересдачу (необходимость сдачи работ сохраняется).

Если студент набрал меньше 50 % от общей суммы баллов, то экзамен будет проходить по билету (2теоретических вопроса + 1 задача).

Если студент набрал больше 50% от суммы максимальной суммы баллов, то сдаёт тестирование на экзамене. + добавляются баллы за активность.

За тест студент может набрать 20 баллов, за которые можно получить:

- От 10 до 14 баллов — оценка «3»;
- От 15 до 19 баллов — оценка «4»;
- 20 баллов — оценка «5».

К баллам за тест прибавляются доп. баллы, которые студент получает в течение семестра по данным критериям:

- Набрано 55-64 балла за семестр — 1 доп.;
- Набрано 65-74 балла за семестр — 2 доп.;
- Набрано 75-84 балла за семестр — 3 доп.;
- Набрано 85-94 балла за семестр — 4 доп.;
- Набрано 95-100 балла за семестр — 5 доп.

В случае, если студент сдал тест на «2», то отправляется на пересдачу без учета доп. баллов.

Данные

Структурированные БД (4 семестр)

СУБД

SQL

- "Классические"
- Нормализация
- "Современные"
- Оптимизация (немного) шардирование и кэширование

Полуструктурированные БД (5 семестр)

NoSQL

- Документориентированные (Maria DB)
- TSBD, Redis, Clickhouse, MONQ, Tinkoff SAGE и т.д.

Неструктурированные БД (магистратура, 2 семестр)

Что «лежит» внутри БД? Где это взять? Как избежать аномалий. Удаление и обновление.

Хранилища

- Блочное хранение
- Витрины данных, формат хранения
- Целостность, состояние
- Что такое XD?

Файловые системы (HDFS, ZFS)

S3, Объектное

State Full, State Less

MapReduce

REST API, Сервисные шины

Управление данными в системе

ПАК

- Единая структура классификации информации в организации
- DAS
- NAS
- SAN

Оптимизация производительности („модели данных + реализация). NoSQL.

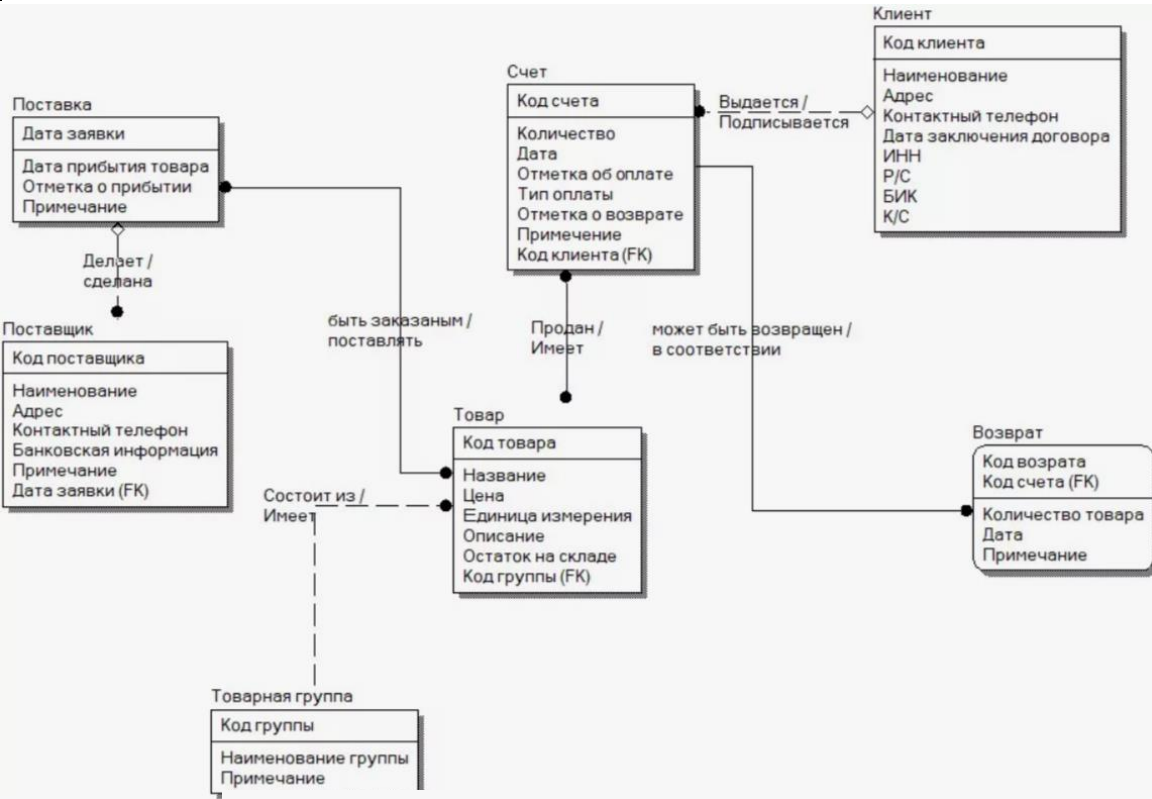
NoSQL.

Архитектура СУБД

Теорема Брюера, репликация, распределенные БД

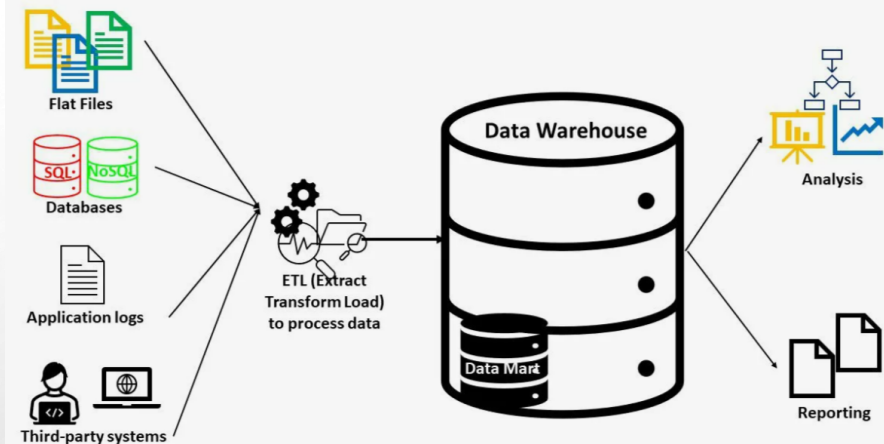
файлы + индекс + язык запросов

Место СУБД в информационной архитектуре предприятия



2 курс. Проектирование баз данных

3 курс. Разработка баз данных



Магистратура Проектирование и разработка баз и хранилищ данных

Тема 6_ Объекты базы данных. Триггеры и курсоры

6.1_Триггеры	12
Триггерные функции	21
6.2. Курсоры	37

Триггеры. Базовый синтаксис

Триггер — это механизм, который позволяет автоматически выполнять определенный код при наступлении определенных событий в базе данных, таких как вставка, обновление или удаление данных. Триггеры связаны с таблицами, представлениями или внешними таблицами и срабатывают до, после или вместо этих операций

```
CREATE [ OR REPLACE ] [ CONSTRAINT ] TRIGGER <имя_триггера>
{ BEFORE | AFTER | INSTEAD OF } { <событие> [ OR ... ] } ON <имя_таблицы>
[ FROM <ссылающаяся_таблица> ]
[ NOT DEFERRABLE | [ DEFERRABLE ] [ INITIALLY IMMEDIATE | INITIALLY DEFERRED ] ]
[ REFERENCING { { OLD | NEW } TABLE [ AS ] <имя> } [ ... ] ]
[ FOR [ EACH ] { ROW | STATEMENT } ]
[ WHEN ( <условие> ) ]
EXECUTE { FUNCTION | PROCEDURE } <имя_функции> ( <аргументы> )
```


Триггеры. Базовый синтаксис

Допускаются события:

INSERT

UPDATE [OF имя_столбца [, ...]]

DELETE

TRUNCATE

- Несколько событий можно указать, добавив между ними слово OR
- Для событий UPDATE можно указать список столбцов:

UPDATE OF имя_столбца1 [, имя_столбца2 ...]

- Триггер для указанных столбцов будет срабатывать, когда эти столбцы перечислены в качестве целевых в списке SET команды UPDATE.
- Для событий INSTEAD OF UPDATE указание списка столбцов не допускается.

Типы триггеров

Типы триггеров, которые могут использоваться для таблиц, представлений и сторонних таблиц:

Когда	Событие	На уровне строк	На уровне оператора
BEFORE	INSERT/UPDATE/DELETE	Таблицы и сторонние таблицы	Таблицы, представления и сторонние таблицы
	TRUNCATE	—	Таблицы и сторонние таблицы
AFTER	INSERT/UPDATE/DELETE	Таблицы и сторонние таблицы	Таблицы, представления и сторонние таблицы
	TRUNCATE	—	Таблицы и сторонние таблицы
INSTEAD OF	INSERT/UPDATE/DELETE	Представления	—
	TRUNCATE	—	—

Типы триггеров

- Триггер можно настроить так, чтобы он срабатывал до операции со строкой (BEFORE) или после её завершения (AFTER), либо вместо операции (INSTEAD OF).
- Триггер с пометкой FOR EACH ROW вызывается один раз для каждой строки, изменяемой в процессе операции.
- Триггер с пометкой FOR EACH STATEMENT вызывается только один раз для конкретной операции, вне зависимости от того, как много строк она изменила
- Триггеры, срабатывающие в режиме INSTEAD OF, должны быть помечены FOR EACH ROW и могут быть определены только для представлений.
- Триггеры BEFORE и AFTER для представлений должны быть помечены FOR EACH STATEMENT.
- Триггеры можно определить для команды TRUNCATE, но только типа FOR EACH STATEMENT

Типы триггеров

- В определении триггера можно указать логическое условие WHEN, которое определит, вызывать триггер или нет.
- Если для одного события определено несколько триггеров одного типа, они будут срабатывать в алфавитном порядке их имён.
- Параметр CONSTRAINT указывает на то, что триггер является триггером ограничения.
- Когда указывается REFERENCING, для триггера собираются переходные отношения, представляющие собой множества строк, включающие все строки, которые были добавлены, удалены или изменены текущим оператором SQL:
 - Это указание допускается только для триггера AFTER, не являющегося триггером ограничения
 - Если это триггер для UPDATE, у него должен отсутствовать список *имён_столбцов*.
 - OLD TABLE (только один раз на триггер, для UPDATE или DELETE триггеров, содержит все строки до изменения/удаления.
 - NEW TABLE (только один раз на триггер, для UPDATE или INSERT триггеров, содержит все строки после изменения/добавления)

Примеры триггеров

Выполнение функции `check_account_update` перед любым изменением строк в таблице `accounts`:

```
CREATE TRIGGER check_update  
  BEFORE UPDATE ON accounts  
  FOR EACH ROW  
  EXECUTE FUNCTION check_account_update();
```

Изменение определения триггера, чтобы данная функция выполнялась только при указании столбца `balance` в качестве целевого столбца команды `UPDATE`:

```
CREATE OR REPLACE TRIGGER check_update  
  BEFORE UPDATE OF balance ON accounts  
  FOR EACH ROW  
  EXECUTE FUNCTION check_account_update();
```


Примеры триггеров

Функция будет выполняться, если значение столбца **balance** в действительности изменилось:

```
CREATE TRIGGER check_update  
  BEFORE UPDATE ON accounts  
  FOR EACH ROW  
  WHEN (OLD.balance IS DISTINCT FROM NEW.balance)  
  EXECUTE FUNCTION check_account_update();
```

где

- OLD — это специальная переменная, содержащая состояние строки до изменения (в случае UPDATE) или перед удалением (в случае DELETE).
- NEW — это другая специальная переменная, содержащая состояние строки после изменения (в случае UPDATE) или вставляемую строку (в случае INSERT).
- IS DISTINCT FROM - определяет, являются ли два значения разными

Примеры триггеров

Вызов функции, ведущей журнал изменений в accounts, но только если что-то изменилось:

```
CREATE TRIGGER log_update  
  AFTER UPDATE ON accounts  
  FOR EACH ROW  
  WHEN (OLD.* IS DISTINCT FROM NEW.*)  
  EXECUTE FUNCTION log_account_update();
```

Выполнение для каждой строки функции view_insert_row, которая будет вставлять строки в таблицы представления:

```
CREATE TRIGGER view_insert  
  INSTEAD OF INSERT ON my_view  
  FOR EACH ROW  
  EXECUTE FUNCTION view_insert_row();
```

Примеры триггеров

Выполнение функции `check_transfer_balances_to_zero` для каждого оператора, проверяющей, что строки `transfer` в совокупности дают нулевой баланс:

```
CREATE TRIGGER transfer_insert  
  AFTER INSERT ON transfer  
  REFERENCING NEW TABLE AS inserted  
  FOR EACH STATEMENT  
  EXECUTE FUNCTION check_transfer_balances_to_zero();
```

Выполнение функции `check_matching_pairs` для каждой строки, проверяющей, что соответствующие пары пунктов изменены синхронно (одним оператором):

```
CREATE TRIGGER paired_items_update  
  AFTER UPDATE ON paired_items  
  REFERENCING NEW TABLE AS newtab OLD TABLE AS oldtab  
  FOR EACH ROW  
  EXECUTE FUNCTION check_matching_pairs();
```

Триггерные функции

Базовый синтаксис создания триггерной функции, написанной на PL/pgSQL:

```
CREATE FUNCTION <имя_функции>()  
RETURNS trigger AS  
$$  
    BEGIN  
        <тело функции>  
        RETURN [NEW|OLD|NULL]  
    END;  
$$  
LANGUAGE plpgsql;
```

Влияние возвращаемого функцией значения на выполнение операций

Операция	Триггер	Возвращаемое значение	Результат
INSERT	BEFORE ROW	NEW	Строка вставляется (с изменениями из триггера).
INSERT	BEFORE ROW	NULL	Строка не вставляется.
UPDATE	BEFORE ROW	NEW	Строка обновляется (с изменениями из триггера).
UPDATE	BEFORE ROW	NULL	Строка не обновляется.
DELETE	BEFORE ROW	OLD	Строка удаляется.
DELETE	BEFORE ROW	NULL	Строка не удаляется.
Любая	AFTER ROW	Любое значение	Игнорируется.
Любая	FOR EACH STATEMENT	NULL	Обязательно!

- Триггерные функции, вызываемые для операторов (FOR EACH STATEMENT), должны возвращать NULL
- Функция, вызываемая для строки, может возвращать следующие значения:
NULL, OLD или NEW

Влияние возвращаемого функцией значения на выполнение операций

RETURN NULL:

- для триггерных функций, выполняемых после операции (AFTER), возвращаемые значения игнорируются;
- если функция, выполняемая до операции со строкой (BEFORE), возвращает NULL, то такая строка пропускается. Операция по изменению, обновлению или удалению с ней не выполняется. При этом выполнение транзакции не прерывается.

RETURN OLD:

- Триггерные функции, выполняемые до (BEFORE) операции DELETE, должны возвращать исходную строку (OLD), если операцию удаления со строкой нужно выполнить. Если же ее нужно пропустить, возвращается NULL.

RETURN NEW:

- Функции, выполняемые до (BEFORE) операции INSERT или UPDATE, должны возвращать новую или обновленную строку, которая будет добавлена в таблицу. Если NEW не вернуть, то строка не будет добавлена/ обновлена

Специальные переменные и константы

NEW record — для триггеров уровня строки содержит новую запись, которая должна стать результатом операции INSERT/UPDATE. В триггерах BEFORE можно изменить эту переменную, чтобы повлиять на данные, которые будут записаны в базу данных.

OLD record — для триггеров уровня строки содержит старую запись, до выполнения UPDATE/DELETE. Недоступна в триггерах INSERT, так как строки еще не существовало.

TG_NAME name — имя триггера.

TG_WHEN text — может содержать BEFORE, AFTER или INSTEAD OF — условие выполнения функции: до, после или вместо операции.

TG_LEVEL text — уровень триггера: для строки (ROW) или операции (STATEMENT).

TG_OP text — операция, для которой запускается триггерная функция: INSERT, UPDATE, DELETE или TRUNCATE.

TG_RELID oid (references pg_class.oid) — object ID таблицы, для которой сработал триггер.

TG_TABLE_NAME name — таблица, для которой сработал триггер.

Изменение и удаление триггеров

Переименование

```
ALTER TRIGGER customer_audit ON customers RENAME TO customer_audit_new;
```

Отключение

```
ALTER TABLE customers DISABLE TRIGGER customer_audit_new;
```

Подключение

```
ALTER TABLE customers ENABLE TRIGGER customer_audit_new;
```

Удаление

```
DROP TRIGGER IF EXIST customer_audit_new ON customers CASCADE;
```

Пример 1. Создание триггера, который вызывает для каждой добавляемой или изменяемой строки таблицы products триггерную функцию price_check(). Триггерная функция проверяет заполнен ли атрибут price

Создаем триггерную функцию:

```
CREATE FUNCTION price_check() RETURNS trigger AS $$  
BEGIN  
    IF NEW.price IS NULL THEN  
        RAISE EXCEPTION 'Стоимость товара % не может быть пустой', NEW.name;  
    END IF;  
    IF NEW.price < 0 THEN  
        RAISE EXCEPTION 'Стоимость товара % должна быть больше нуля', NEW.name;  
    END IF;  
    RETURN NEW;  
END; $$  
LANGUAGE plpgsql;
```

Создаем триггер:

```
CREATE OR REPLACE TRIGGER price_check  
BEFORE UPDATE OF price OR INSERT ON products  
FOR EACH ROW  
EXECUTE FUNCTION price_check();
```

Тестирование: 1. Вставка строки:

```
INSERT INTO products(name,  
quantity_in_stock, price, production_date,  
expiration_date,company_id)  
VALUES('Сахар', 10, NULL, '2025-08-09', NULL, 2);
```

ERROR: Стоимость товара Сахар не может быть пустой

????????: PL/pgSQL function price_check() line 4 at RAISE

SQL state: P0001

Context: PL/pgSQL function price_check() line 4 at RAISE

2. Изменение строки:

```
UPDATE products SET price = NULL WHERE id = 1;
```

ERROR: Стоимость товара Молоко 3.2% не может быть пустой

????????: PL/pgSQL function price_check() line 4 at RAISE

SQL state: P0001

Context: PL/pgSQL function price_check() line 4 at RAISE

Пример 2. Логирование изменений в таблице покупателей:

-- Создаем таблицу для хранения изменений

```
CREATE TABLE customer_audit (  
    user_id          INT,  
    first_name       text,  
    last_name        text,  
    phone_number      text,  
    change_date      timestamp  
);
```

-- Создаем триггерную функцию

```
CREATE OR REPLACE FUNCTION customer_audit()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    IF OLD.* IS DISTINCT FROM NEW.* THEN
```

```
        INSERT INTO customer_audit(user_id, first_name,    last_name,  
                                   phone_number, change_date)
```

```
            VALUES(OLD.customer_id, OLD.first_name,
```

```
                   OLD.last_name, OLD.phone_number, now());
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END; $$ LANGUAGE plpgsql;
```

-- Создаем триггер уровня строки:

```
CREATE OR REPLACE TRIGGER customer_audit BEFORE UPDATE ON customers
FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*) EXECUTE PROCEDURE customer_audit();
```

1. Исходная таблица **SELECT * FROM customers;**

customer_id 	first_name 	last_name 	email 	phone_number 
1	Иван	Иванов	ivan@example.com	+79001234567
2	Петр	Петров	petr@example.com	[null]

2. Изменяем данные:

```
UPDATE customers SET phone_number =
'+79094562345' WHERE customer_id = 1;
```

3. Измененные данные: **SELECT * FROM customers;**

customer_id 	first_name 	last_name 	email 	phone_number 
2	Петр	Петров	petr@example.com	[null]
1	Иван	Иванов	ivan@example.com	+79094562345

4. Журнал логов:

```
SELECT * FROM customer_audit;
```

id 	first_name 	last_name 	phone_number 	change_date 
1	Иван	Иванов	+79001234567	2025-08-18 00:03:48.210531

Пример 3. Разработать триггер в PostgreSQL, который автоматизирует процесс обновления информации о товарах в таблице products, чтобы избежать дублирования данных при поступлении новых партий товара

```
CREATE OR REPLACE FUNCTION update_quantity_product()
RETURNS TRIGGER AS $$
DECLARE
    product_id INTEGER;
BEGIN
    SELECT id INTO product_id FROM products WHERE name = NEW.name
    AND price = NEW.price AND company_id = NEW.company_id AND production_date = NEW.production_date;
    IF product_id IS NOT NULL THEN
        UPDATE products
        SET quantity_in_stock = quantity_in_stock + NEW.quantity_in_stock WHERE id = product_id;
        RETURN NULL;
    ELSE
        RETURN NEW;
    END IF; END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER before_insert_product
BEFORE INSERT ON products
FOR EACH ROW
EXECUTE FUNCTION update_quantity_product();
```

```
INSERT INTO products (name, price, company_id,
quantity_in_stock, production_date)
VALUES ('Греча', 95.00, 2, 20, '2025-06-01');
```

```
INSERT INTO products (name, price,
company_id, quantity_in_stock, production_date)
VALUES ('Бананы', 150.00, 2, 100, '2015-08-09');
```

id [PK] integer	name character varying (255)	quantity_in_stock integer	price numeric (10,2)
8	Греча	30	95.00

8	Греча	50	95.00	2025-06-01	[null]
22	Бананы	100	150.00	2015-08-09	[null]

Пример 4. Разработать набор триггеров в PostgreSQL, которые будут автоматически обновлять информацию о заказах и остатках товаров в магазине при добавлении или удалении товаров из заказа.

```
CREATE OR REPLACE FUNCTION
before_check_quantity() RETURNS TRIGGER AS $$
DECLARE
    q_stock INTEGER;  product_price DECIMAL(10, 2);
BEGIN
    SELECT quantity_in_stock INTO q_stock
    FROM products  WHERE id = NEW.product_id;

    IF q_stock < NEW.quantity THEN
        RAISE EXCEPTION 'Недостаточно товара
            на складе. Доступно: %', q_stock;
    END IF;
```

```
SELECT price INTO product_price
FROM products
WHERE id = NEW.product_id;
NEW.unit_price := product_price;
```

```
UPDATE products
SET quantity_in_stock = quantity_in_stock - NEW.quantity
WHERE id = NEW.product_id;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION calculate_order_total()
RETURNS TRIGGER AS $$
DECLARE
    total DECIMAL(10, 2);
BEGIN
    SELECT SUM(quantity * unit_price) INTO total
        FROM order_items WHERE order_id = NEW.order_id;
    UPDATE orders SET total_amount = total
        WHERE order_id = NEW.order_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER insert_order_item
BEFORE INSERT ON order_items
FOR EACH ROW
EXECUTE FUNCTION before_check_quantity();
```

```
CREATE OR REPLACE TRIGGER after_insert_order_item
AFTER INSERT ON order_items
FOR EACH ROW
EXECUTE FUNCTION calculate_order_total();
```



```
CREATE OR REPLACE FUNCTION recalculate_order_total()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE orders
    SET total_amount = (
        SELECT COALESCE(SUM(quantity * unit_price), 0)
        FROM order_items WHERE order_id = OLD.order_id
    )
    WHERE order_id = OLD.order_id;
    UPDATE products
    SET quantity_in_stock = quantity_in_stock + OLD.quantity
    WHERE id = OLD.product_id;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER after_delete_order_item
AFTER DELETE ON order_items
FOR EACH ROW
EXECUTE FUNCTION recalculate_order_total();
```

Тестирование:

INSERT INTO orders (customer_id) VALUES (2);

4	4	2	2025-01-23	200.00
5	6	2	2025-08-20	0.00

INSERT INTO order_items (order_id, product_id, quantity) VALUES (6, 5, 10);

4	Яйца куриные	200	120.00
5	Вода питьевая	300	25.00
6	Рис длинозерный	80	90.00

SELECT * FROM orders WHERE order_id = 6;

order_id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)
6	2	2025-08-20	250.00

SELECT * FROM products WHERE id =5;

id integer	name character varying (255)	quantity_in_stock integer	price numeric (10,2)
5	Вода питьевая	290	

INSERT INTO order_items (order_id, product_id, quantity) VALUES (6, 5, 300);

ERROR: Недостаточно товара на складе. Доступно: 290

PL/pgSQL function before_check_quantity() line 13 at RAISE

Порядок срабатывания триггеров

- Если для одного и того же события в одном отношении определено более одного триггера, триггеры сработают в алфавитном порядке по имени триггера.
- В случае триггеров BEFORE и INSTEAD OF строка, возвращаемая каждым триггером, становится входными данными для следующего триггера.
- Если какой-либо триггер BEFORE или INSTEAD OF возвращает NULL, операция останавливается и последующие триггеры для этой строки не срабатывают.

Каскадные триггеры

- Если триггерная функция выполняет команды SQL, эти команды могут снова запустить триггеры. Это поведение известно как каскадные триггеры.
- Прямого ограничения на количество каскадных уровней нет.
- Каскады могут вызывать рекурсивный вызов одного и того же триггера. Например, триггер INSERT может выполнить команду, которая вставляет дополнительную строку в ту же таблицу, вызывая повторное срабатывание триггера INSERT.
- Ответственность за предотвращение бесконечной рекурсии в таких случаях лежит на программисте.
- Если триггерная функция завершится с ошибкой, PostgreSQL откатывает всю транзакцию.

Использование триггеров:

1. Триггеры могут гарантировать соблюдение сложных бизнес-правил и ограничений, которые сложно реализовать с помощью стандартного функционала базы данных.
2. Часто при изменении данных в одной таблице необходимо автоматически обновлять данные в связанных таблицах. Триггеры могут выполнять такие обновления, обеспечивая согласованность данных.
3. В некоторых случаях значения полей должны автоматически рассчитываться на основе других полей. Триггеры могут выполнять такие вычисления и обновлять значения полей при изменении данных.
4. Триггеры применяют для ведения истории изменения, они могут автоматически создавать записи в журнале при вставке, обновлении или удалении данных, фиксируя информацию о том, кто и когда внёс изменения.
5. Триггеры могут проверять права доступа и ограничивать операции на уровне базы данных. Например, можно создать триггеры, которые предотвращают удаление критически важных записей или ограничивают доступ к определенным данным для выбранных пользователей.

Курсоры. Основные понятия

Курсор - объект, который связан с данными, полученными с помощью оператора SQL, и позволяет осуществлять доступ к каждой отдельной строке.

Результирующий набор курсора - множество строк, которое получено в результате запроса и с которым связывается курсор;

Позиция курсора - указатель на одну из строк результирующего набора.

В PostgreSQL курсоры позволяют обрабатывать результаты запросов построчно, что полезно для сложных операций, когда нужно выполнять действия над каждой строкой, запоминая информацию о предыдущих строках.

Команды управления курсорами

- DECLARE: Объявляет курсор.
- OPEN: Открывает курсор.
- FETCH: Извлекает строки из курсора.
- CLOSE: Закрывает курсор.
- MOVE: Перемещает курсор по набору данных.
- DEALLOCATE: Удаляет курсор.

Доступ к курсорам в PL/pgSQL осуществляется через курсорные переменные, которые всегда имеют специальный тип данных `refcursor`.

PostgreSQL поддерживает два типа курсоров: неявные (создаются автоматически при использовании предложения `FOR` в `SELECT`) и явные (объявляются с помощью `DECLARE`).

Курсорные переменные

Способы создания курсорной переменной:

1) объявить её как переменную типа refcursor.

2) использовать синтаксис объявления курсора:

имя [[NO] SCROLL] CURSOR [(аргументы)] FOR запрос.

При этом аргументы могут включать параметры, необходимые для выполнения запроса.

DECLARE

 curs1 refcursor;

 curs2 CURSOR FOR SELECT * FROM tenk1;

 curs3 CURSOR (key integer) FOR SELECT * FROM tenk1 WHERE
unique1 = key;

Открытие курсора

В PL/pgSQL есть три формы оператора OPEN, две из которых используются для несвязанных курсорных переменных, а третья для связанных:

1) OPEN несвязанная_переменная_курсора [[NO] SCROLL] FOR query

Пример: OPEN curs1 FOR SELECT * FROM foo WHERE key = mykey;

2) OPEN несвязанная_переменная_курсора [[NO] SCROLL] FOR EXECUTE строка_запроса [USING выражение [, ...]];

Пример: OPEN curs1 FOR EXECUTE format('SELECT * FROM %I WHERE col1 = \$1',tabname)
USING keyvalue;

Открытие курсора

3) OPEN связанная_переменная_курсора [([имя_аргумента :=] значение_аргумента [, ...])];

Примеры:

```
OPEN curs2;
```

```
OPEN curs3(42);
```

```
OPEN curs3(key := 42);
```

```
DECLARE
```

```
    key integer;
```

```
    curs4 CURSOR FOR SELECT * FROM tenk1 WHERE unique1 = key;
```

```
BEGIN
```

```
    key := 42;
```

```
    OPEN curs4;
```

Связанные курсорные переменные можно использовать с циклом FOR без явного открытия курсора. Цикл FOR откроет курсор, а затем закроет его по завершении цикла.

Извлечение строки из курсора

FETCH [направление { FROM | IN }] курсор INTO цель;

Направление может быть любым допустимым в SQL-команде FETCH вариантом, кроме тех, что извлекают более одной строки:

- **FETCH NEXT:** извлекает следующую строку.
- **FETCH PRIOR:** извлекает предыдущую строку.
- **FETCH FIRST:** извлекает первую строку.
- **FETCH LAST:** извлекает последнюю строку.
- **FETCH ABSOLUTE n:** извлекает строку с номером n.
- **FETCH RELATIVE n:** извлекает строку на n позиций относительно текущей.

Примеры:

```
FETCH curs1 INTO rowvar;
```

```
FETCH curs2 INTO foo, bar, baz;
```

```
FETCH LAST FROM curs3 INTO x, y;
```

```
FETCH RELATIVE -2 FROM curs4 INTO x;
```

Извлечение строки из курсора

Пример: извлечение первого товара из таблицы products

```
DO $$
```

```
DECLARE
```

```
    curs1 CURSOR FOR SELECT id, name FROM products;
```

```
    rowvar RECORD;
```

```
BEGIN
```

```
    OPEN curs1;
```

```
    FETCH curs1 INTO rowvar;
```

```
    IF rowvar IS NOT NULL THEN
```

```
        RAISE NOTICE 'id: %, name: %', rowvar.id, rowvar.name;
```

```
    END IF;
```

```
    CLOSE curs1;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```


Перемещение курсора

MOVE [направление { FROM | IN }] курсор;

Примеры: MOVE curs1;

MOVE LAST FROM curs3;

MOVE RELATIVE -2 FROM curs4;

MOVE FORWARD 2 FROM curs4;

Когда курсор позиционирован на строку таблицы, эту строку можно изменить или удалить при помощи курсора:

UPDATE таблица SET ... WHERE CURRENT OF курсор;

DELETE FROM таблица WHERE CURRENT OF курсор;

Пример: UPDATE foo SET dataval = myval WHERE CURRENT OF curs1

Возврат курсора из функции

Создание тестовой таблицы:

```
CREATE TABLE test (col text); INSERT INTO test VALUES ('123');
```

Функция, возвращающая курсор:

```
CREATE FUNCTION reffunc(refcursor) RETURNS refcursor AS $$  
BEGIN  
    OPEN $1 FOR SELECT col FROM test;  
    RETURN $1;  
END; $$ LANGUAGE plpgsql;
```

Вызов функции

```
BEGIN;  
SELECT reffunc('funcursor'); FETCH ALL IN funcursor;  
COMMIT;
```

Возврат нескольких курсоров

```
CREATE FUNCTION myfunc(refcursor, refcursor) RETURNS SETOF refcursor AS $$  
BEGIN  
    OPEN $1 FOR SELECT * FROM table_1;  
    RETURN NEXT $1;  
    OPEN $2 FOR SELECT * FROM table_2;  
    RETURN NEXT $2;  
END;  
$$ LANGUAGE plpgsql;
```

```
BEGIN;  
SELECT * FROM myfunc('a', 'b');  
FETCH ALL FROM a;  
FETCH ALL FROM b;  
COMMIT;
```

Перебор строк, возвращаемых курсором

Цикл FOR позволяет перебирать строки, возвращённые курсором:

[<<метка>>]

FOR переменная-запись IN связанная_переменная_курсора [([имя_аргумента :=]
значение_аргумента [, ...])] LOOP

операторы

END LOOP [метка];

Пример:

DO \$\$

DECLARE

my_cursor CURSOR FOR SELECT * FROM customers;

user_row customers%ROWTYPE;

BEGIN

FOR user_row IN my_cursor LOOP

RAISE NOTICE 'ID: %, Name: %', user_row.customer_id, user_row.last_name;

END LOOP;

END;

\$\$ LANGUAGE plpgsql;

Чувствительные и нечувствительные курсоры

По умолчанию курсоры в PostgreSQL считаются *нечувствительными*. Это означает, что если данные в базовых таблицах, на которых основан курсор, изменяются *после* открытия курсора, эти изменения *не* будут отражены в результатах, извлекаемых из курсора.

Для того, чтобы сделать курсор чувствительным необходимо указать FOR UPDATE при объявлении курсора. Это блокирует строки, выбранные курсором, чтобы другие транзакции не могли их изменить до тех пор, пока работа с курсором не будет закончена, а транзакция не зафиксирована.

Пример:

```
DECLARE curs1 CURSOR FOR SELECT id, name FROM mytable FOR UPDATE;  
-- Обновляем данные в таблице (изменения будут видны через курсор, если  
использовать FETCH)  
UPDATE mytable SET name = 'New Name' WHERE id = 1;  
-- Фиксируем транзакцию (освобождаем блокировки)  
COMMIT;
```

Проанализируйте приведенный код. Предположим, что в таблице employees есть запись с id=1 и salary=50000. Что произойдет при выполнении приведенного ниже запроса UPDATE?

```
CREATE OR REPLACE TRIGGER before_employee_update  
BEFORE UPDATE ON employees FOR EACH ROW  
WHEN (NEW.salary < OLD.salary)  
EXECUTE FUNCTION prevent_salary_decrease();
```

```
CREATE OR REPLACE FUNCTION prevent_salary_decrease()  
RETURNS TRIGGER AS $$  
BEGIN  
    RAISE EXCEPTION 'Уменьшение зарплаты запрещено!';  
END;  
$$ LANGUAGE plpgsql;
```

```
UPDATE employees SET salary = 45000 WHERE id = 1;
```


Проанализируйте приведенный код. Предположим, что в таблице employees есть запись с id=1 и salary=50000. Что произойдет при выполнении приведенного ниже запроса UPDATE?

```
CREATE OR REPLACE TRIGGER before_employee_update
BEFORE UPDATE ON employees FOR EACH ROW
WHEN (NEW.salary < OLD.salary)
EXECUTE FUNCTION prevent_salary_decrease();

CREATE OR REPLACE FUNCTION prevent_salary_decrease()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Уменьшение зарплаты запрещено!';
END;
$$ LANGUAGE plpgsql;
```

```
UPDATE employees SET salary = 45000 WHERE id = 1;
```

Выполнение запроса завершится ошибкой с сообщением "Уменьшение зарплаты запрещено!"

Условие WHEN (NEW.salary < OLD.salary) проверяет, уменьшается ли зарплата.

Проанализируйте приведенный код. Предположим, что в таблице orders есть запись с id=1, status='pending' и total_amount=1000. Что произойдет при выполнении приведенного ниже запроса UPDATE?

```
CREATE OR REPLACE TRIGGER before_order_update
BEFORE UPDATE ON orders FOR EACH ROW
WHEN (OLD.status = 'shipped' AND NEW.status != 'shipped')
EXECUTE FUNCTION prevent_status_change();
```

```
CREATE OR REPLACE FUNCTION prevent_status_change()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Нельзя изменить статус отправленного заказа!';
END;
$$ LANGUAGE plpgsql;
```

```
UPDATE orders SET status = 'delivered' WHERE id = 1;
```

Проанализируйте приведенный код. Предположим, что в таблице `orders` есть запись с `id=1`, `status='pending'` и `total_amount=1000`. Что произойдет при выполнении приведенного ниже запроса `UPDATE`?

```
CREATE OR REPLACE TRIGGER before_order_update
BEFORE UPDATE ON orders FOR EACH ROW
WHEN (OLD.status = 'shipped' AND NEW.status != 'shipped')
EXECUTE FUNCTION prevent_status_change();
```

```
CREATE OR REPLACE FUNCTION prevent_status_change()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Нельзя изменить статус!';
END;
$$ LANGUAGE plpgsql;
```

```
UPDATE orders SET status = 'delivered' WHERE id = 1;
```

Запрос выполнится успешно, триггер НЕ сработает.

Условие `WHEN (OLD.status = 'shipped' AND NEW.status != 'shipped')` проверяет попытку изменить статус отправленного заказа. Но исходный статус равен `'pending'`, а не `'shipped'`, поэтому условие не выполняется, и триггер не активируется.

Спасибо за внимание!